Fixed Geometric Routing from Angular Structure in Normalized Language Representations

Casey Allard
Independent Researcher

Abstract

Sparse expert models usually obtain routing concentration by learning a gate and regularizing that gate during training [Shazeer et al., 2017, Lepikhin et al., 2021, Fedus et al., 2022, Lewis et al., 2021]. This work studies a narrower question: whether some routing structure can be supplied *before* gate learning, directly from the angular organization of normalized language representations. We evaluate a fixed routing map built from a 4D coordinate chart, a Hopf-style angular parameterization, and a transport-corrected phase coordinate. On a semantic proxy derived from structured WikiText-2 embeddings, the resulting routing footprint is sublinear in routing budget, with fitted law $\hat{e} = 2.957K^{0.572}$ over $K = 25$–400, and the advantage persists through $K = 5000$. The effect weakens under column permutation of the embeddings and is stronger than an adaptive Euclidean K-means control on the same proxy, which constrains the interpretation to structured angular signal rather than generic sparsity. In a two-layer language model with 64 experts, the same fixed routing scaffold remains usable: on both Penn Treebank and WikiText-2 it stays within about 8% of a learned top-1 baseline while removing the learned gate matrix. The central limitation is also the central negative result: after end-to-end training, Hopf routing is not measurably better than a randomly permuted fixed routing map. The evidence therefore supports a narrow conclusion. Angular structure in normalized language representations can support concentrated fixed routing, and that scaffold can transfer to a toy trainable model, but the present results do not establish a geometry-specific quality advantage in end-to-end language-model training.

1 Introduction

Conditional computation in language models is usually mediated by a learned router. In sparse mixture-of-experts systems, a gating network assigns each token to one or more experts, and additional training machinery is then used to keep that routing stable, balanced, and numerically tractable [Shazeer et al., 2017, Lepikhin et al., 2021, Fedus et al., 2022, Lewis et al., 2021]. That recipe is effective, but it entangles routing behavior with gate optimization. It therefore leaves open a simpler preliminary question: how much routing structure is already present in the geometry of the representations being routed?

This paper studies a fixed alternative. Each normalized representation is projected into a fixed four-dimensional chart, converted into a small set of angular coordinates, and assigned to a routing sector with no learned gate matrix. The motivation is empirical. Prior work on contextual representation geometry has shown that trained language-model embeddings are often strongly anisotropic rather than close to uniform on the sphere [Ethayarajh, 2019]. More recent angular-compression work exploits the same observation by destroying angular structure with rotation before quantization [Zandieh et al., 2025]. Here the engineering choice is the opposite. If normalized representations occupy angular coordinates non-uniformly, then a fixed angular partition should receive non-uniform traffic and may therefore concentrate routing without any learned gate.

The paper makes one narrow claim with two layers of evidence. The first layer is a static semantic proxy built from structured embeddings, where the routing map itself can be evaluated without expert co-adaptation.

The second layer is a small trainable language model, where the same fixed routing scaffold can be tested under end-to-end learning. The main positive result is that the proxy exhibits a sublinear routing footprint and that the same fixed scaffold partially transfers to a toy trainable model. The main negative result is that, once experts are trained end to end, the specific Hopf-derived assignment is not better than a randomly permuted fixed routing map.

That null result matters for the paper's scope. The evidence supports fixed geometric routing as a usable scaffold and as a concrete consequence of angular structure in normalized language representations. It does not support a broader claim that this particular geometry is uniquely superior to other fixed assignments after training, nor does it justify claims about replacing modern large-scale sparse routing systems.

2 Problem Setting and Scope

The work is positioned against learned sparse routing, but it does not attempt a full modern MoE comparison. Learned routing systems jointly optimize two objects: a router and a bank of experts. Once those two objects can co-adapt, it becomes difficult to tell whether routing concentration comes from the geometry of the inputs, from the learned gate, or from compensation by the experts themselves. The experiments here separate those questions.

The first setting is a semantic proxy built from static WikiText-2-derived embeddings [Merity et al., 2016]. This setting removes gate learning entirely and therefore tests the routing map as a geometric object. It is the right place to ask whether structured language representations occupy a fixed angular partition unevenly enough to produce concentrated routing. The second setting is a two-layer transformer-like language model evaluated on Penn Treebank [Marcus et al., 1993] and WikiText-2. This is not intended as a production-scale sparse-transformer benchmark. Its role is narrower: to test whether the same fixed routing scaffold remains usable after experts are trained end to end.

The paper uses one primary response variable in both settings: the *effective routing footprint*, meaning the effective number of routes used under the empirical routing distribution. On the semantic proxy this is the effective number of occupied sectors. In the language model it is the effective number of active expert paths. Lower values at fixed routing budget indicate more concentrated traffic.

This framing intentionally excludes several broader themes that exist elsewhere in the repository: product-manifold architectural framing, spectral operators, sparse-event execution, and large projected hardware savings. Those ideas may motivate future work, but they are not needed for the claim tested here and are not supported directly by the evidence reported in this article.

3 Fixed Geometric Routing

3.1 Mechanism overview

The mechanism is fixed and deterministic. Given a normalized representation $\mathbf{x} \in \mathbb{R}^D$, select a fixed four-dimensional coordinate chart, summarize that chart with a small set of angular variables, and quantize those variables into one of at most K routing sectors. If the input distribution is close to uniform in those coordinates, then increasing K should activate routes almost linearly. If the input distribution is anisotropic, traffic should collect in a smaller subset of routes. The method is therefore a direct test of whether normalized language representations carry stable angular structure that a fixed partition can exploit.

3.2 Fixed 4D chart and Hopf-style coordinates

Figure 1 summarizes the construction. Let the normalized input be $\mathbf{x} \in \mathbb{R}^D$. The routing rule reads a fixed coordinate tuple $(3, 65, 2, 21)$ and writes the projected coordinates as (a, b, c, d). The role of this chart

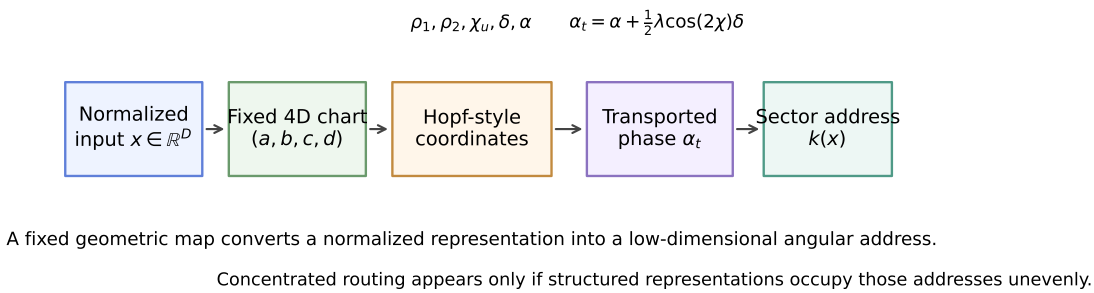

Figure 1: Fixed geometric routing used in this study. A normalized representation is read through a fixed 4D chart, converted into Hopf-style angular coordinates, corrected on the phase coordinate by a transport term, and discretized into a sector address. The routing map is fixed; any concentration must therefore come from the occupancy pattern of the representations rather than from learned gate adaptation.

is practical: it creates a small, stable coordinate system in which angular structure can be parameterized explicitly without introducing another learned module.

From (a, b, c, d) the method computes two planar radii and three angular quantities,

$$\rho_1 = \sqrt{a^2 + b^2}, \qquad \rho_2 = \sqrt{c^2 + d^2}, \tag{1}$$

$$\chi_u = \frac{\rho_2^2}{\rho_1^2 + \rho_2^2}, \qquad \delta = \arctan 2(b, a) - \arctan 2(d, c), \tag{2}$$

$$\alpha = \tfrac{1}{2}(\arctan 2(b, a) + \arctan 2(d, c)) . \tag{3}$$

These variables separate three different aspects of the projection. The coordinate χ_u records how mass is split between the two 2D planes, δ records the relative angle between those planes, and α records a mean phase. The point is not to claim that this is the unique optimal representation of the geometry. The point is that it provides a concrete, low-dimensional address that can be fixed across all experiments.

3.3 Transport-corrected phase and sectorization

The canonical routing rule perturbs the phase coordinate according to

$$\chi = \arcsin\left(\frac{\rho_2}{\sqrt{\rho_1^2 + \rho_2^2}}\right), \qquad \alpha_t = \alpha + \tfrac{1}{2}\lambda \cos(2\chi)\,\delta, \tag{4}$$

where λ is fixed. Mechanically, this transport term shifts the mean phase according to where the point lies in the low-dimensional geometry. Two inputs with similar raw phase can therefore receive different routing addresses if their relative-angle structure differs. The transport term is part of the canonical routing rule used in the evidence base; the paper does not isolate it as a separate contribution.

Given a routing budget K, the coordinate triple $(\chi_u, \delta, \alpha_t)$ is discretized into product bins $(k_\chi, k_\delta, k_\alpha)$ with total budget at most K. The resulting triplet index is the sector address. Because the map is fixed, any reduction in the effective routing footprint must come from the occupancy pattern of the data rather than from gate learning.

3.4 Why this construction can concentrate routing

The proxy-side claim is not that every fixed partition will concentrate routing. It is that a fixed partition aligned with angular structure can do so when the representation cloud is anisotropic. That distinction is important. A structure-destroying control should weaken the concentration effect, and a generic Euclidean comparator need not match it even if it adapts cluster centroids in a much higher-dimensional space. Those two tests are built directly into the empirical sections below.

4 Experimental Design

4.1 Semantic proxy

The semantic proxy is built from a WikiText-2-derived PPMI-SVD embedding set. This proxy is static: there is no end-to-end language-model backpropagation and no expert co-adaptation. Its purpose is to test the routing map itself. Routing budget is swept from $K = 25$ to $K = 400$ for the canonical scaling law and extended to $K = 5000$ for the large-budget persistence check. The proxy analyses report the effective routing footprint together with sector entropy and peak sector mass $p_{\max}$, which help distinguish genuine concentration from superficial occupancy effects.

The key controls are also defined at the proxy level. In the first control, the same embeddings are column-permuted before routing, which preserves marginal statistics while destroying the structured alignment between coordinates and semantic signal. In the second, fixed Hopf routing is compared against adaptive Euclidean K-means on the same structured proxy. If the main effect is genuinely geometric rather than generic, it should weaken under permutation and need not be reproduced by the Euclidean comparator.

4.2 Trainable language model

The trainable setting uses a two-layer transformer-like language model with hidden dimension 128, four attention heads, vocabulary size 5000, sequence length 32, 64 experts, and expert dimension 128. All conditions train for 4000 optimization steps with two seeds per dataset. The reported datasets are Penn Treebank and WikiText-2.

Three routing conditions are central. Baseline is learned top-1 routing without an auxiliary load-balancing loss. Hopf removes the learned gate and uses the fixed geometric sector map. Permuted keeps the same fixed-routing scaffold but randomly permutes the sector assignments. This null condition is crucial because it preserves sparsity and parameter count while removing any geometry-specific assignment pattern.

The choice of Baseline deserves emphasis. The question here is whether a fixed routing map can replace a naturally concentrated learned sparse regime. An auxiliary load-balancing objective pushes learned routing toward near-uniform usage and therefore changes the comparison target. That auxiliary-loss control is reported in the appendix as support for comparator choice, not as a headline result.

5 Semantic Proxy Results

5.1 Sublinear routing footprint

The semantic proxy provides the paper's strongest evidence. Figure 2 shows the effective routing footprint as a function of routing budget. Dense routing scales linearly by construction. Fixed geometric routing scales much more slowly: over the anchor range $K = 25$–400, the fitted law is

$$\hat{e} = 2.957K^{0.572}. \tag{5}$$

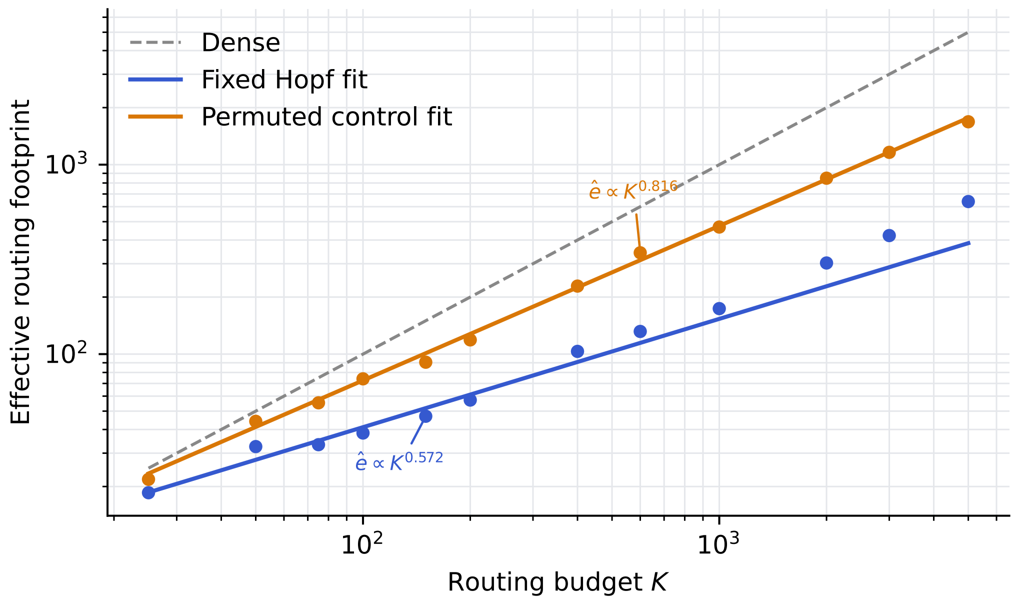

Figure 2: Effective routing footprint on the semantic proxy as a function of routing budget. Fixed geometric routing grows sublinearly, whereas dense routing grows linearly and the permuted fixed-routing control grows substantially more steeply. The large-budget extension indicates that the gap is not a small-K artifact.

The corresponding permuted fixed-routing control scales with exponent 0.814, much closer to linear. The effect is not confined to the anchor range. Extending the same analysis to $K = 5000$ yields a refitted Hopf exponent of approximately 0.657, and the advantage over the permuted control reaches 2.21× at $K = 400$ and 2.64× at $K = 5000$.

This result should be read as a statement about occupancy, not about trained language-model quality. On the proxy, the routing map itself is the object under test, and the main question is whether structured representations populate a fixed angular partition unevenly enough to reduce the effective number of active routes.

5.2 Controls that constrain the interpretation

The main scaling law would be much less informative without controls. Figure 3 makes the two most important ones visible. First, the effect depends on structure in the embeddings. When the same proxy data are column-permuted before routing, peak sector mass drops from 0.090 to 0.061 and the entropy of the routing distribution rises. Second, the effect is not reproduced equally well by a generic adaptive Euclidean comparator. On the same structured proxy, fixed Hopf routing reaches sector $p_{\max} = 0.087$, whereas adaptive K-means reaches 0.072.

Together, these controls narrow the interpretation of Figure 2. The effect is not generic route sparsity, not a trivial consequence of fixed assignment, and not matched by a high-dimensional adaptive Euclidean clustering baseline. Appendix B provides the detailed tables, and Appendix B.1 shows that the fitted exponent remains essentially unchanged across L1, L2, L3, and L4 normalization, with span below 0.000.

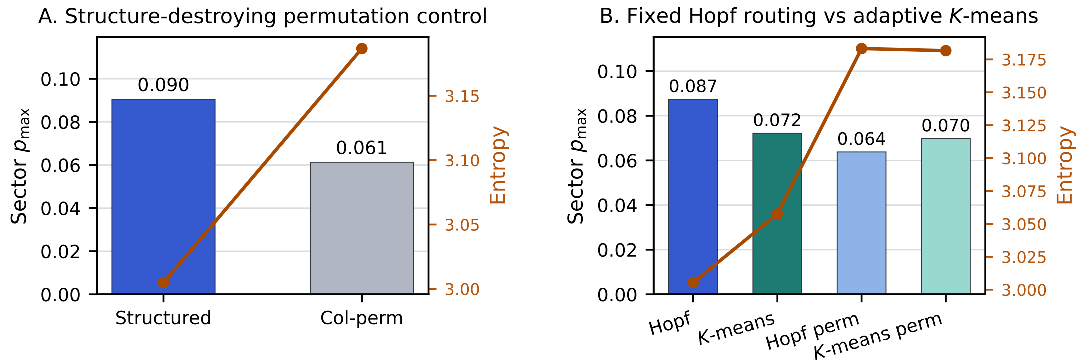

Figure 3: Two controls that anchor the proxy result. Left: concentration weakens when the structured embeddings are column-permuted before routing. Right: fixed Hopf routing is more concentrated than adaptive Euclidean K-means on the same structured proxy, and both routing families weaken under permutation controls.

Table 1: Main trainable-model result. Fixed geometric routing remains usable in a two-layer language model on both PTB and WikiText-2, but it does not improve over the permuted fixed-routing null.

Dataset	Condition	Val. PPL	Eff. paths	$64/\mathrm{eff}_b$
PTB	BASELINE	152.26	43.19	1.48×
PTB	HOPF	164.54	45.96	1.39×
PTB	PERMUTED	164.41	45.81	1.40×
WT2	BASELINE	105.85	35.37	1.81×
WT2	HOPF	114.45	40.94	1.56×
WT2	PERMUTED	114.48	42.28	1.51×

6 Results in a Trainable Language Model

6.1 Partial transfer to PTB and WikiText-2

Table 1 summarizes the trainable language-model results. On Penn Treebank, fixed Hopf routing achieves a HOPF/BASELINE validation-perplexity ratio of 1.081. On WikiText-2, the same ratio is 1.081. Across both datasets, the fixed scaffold remains within about 8% of the learned top-1 baseline while removing the learned gate matrix.

The routing-efficiency side of the result is modest but consistent. Hopf uses 46.0 effective expert paths on PTB and 40.9 on WikiText-2 out of 64 available experts. That is weaker than the proxy-side concentration effect, but it does show that the fixed scaffold can function inside a trainable model without catastrophic collapse.

6.2 The null result against permuted routing

The most important negative result in the paper is that the specific Hopf-derived assignment is not better than a randomly permuted fixed routing map after end-to-end training. In both datasets, Hopf and Permuted are effectively indistinguishable on validation perplexity: the difference is +0.13 on Penn Treebank and -0.03 on WikiText-2. Appendix C provides the compact null-result summary together with the cross-dataset support tables.

This null result changes the meaning of the trainable evidence. The proxy demonstrates that structured representations interact nontrivially with the Hopf-derived sector map. Once the experts are allowed to adapt, however, they can co-adapt to whichever fixed scaffold they receive. The trainable evidence therefore supports a claim about fixed routing as a usable scaffold, not a claim that Hopf geometry is uniquely superior after learning.

6.3 Why the learned comparator is no-aux top-1 routing

The learned comparator is intentionally a top-1 gate without an auxiliary load-balancing loss. That choice follows from the question under study. The comparison is meant to test whether a fixed routing map can substitute for a naturally concentrated sparse regime, not whether it can imitate a deliberately balance-forced router. In the auxiliary-loss control, the balance objective drives learned routing toward near-uniform usage, with 62.8 effective paths out of 64. That table is included in Appendix C. It supports the choice of comparator, but it is not the headline result.

7 Discussion and Limitations

The evidence supports a limited but nontrivial conclusion. On a static semantic proxy, a fixed routing map derived from angular structure in normalized language representations yields a sublinear routing footprint, and that advantage persists to large routing budgets. The same fixed scaffold can also be inserted into a toy trainable language model and remain within a modest perplexity margin of a learned top-1 baseline.

The limitations are equally important. The strongest evidence is proxy-side rather than trainable. The trainable architecture is small: two layers, 64 experts, short training runs, and FFN routing only. Most importantly, the geometry-specific advantage disappears under expert co-adaptation. Once the model is trained end to end, Hopf routing is not better than a randomly permuted fixed routing scaffold on quality. That result is substantive, not incidental. It is why the paper argues for fixed geometric routing as a constrained first result rather than as a general replacement for learned sparse routing.

Recent angular-compression results make the broader motivation concrete: angular organization in normalized representations appears to have engineering consequences beyond a single use case. The evidence here, however, supports only the routing consequence tested directly in this article. Larger architectural implications remain conjectural unless they are validated in larger trainable systems.

8 Conclusion

This work isolates and tests a narrow proposition: angular structure in normalized language representations can support a fixed routing map. The evidence supports that proposition in two ways. First, on a static semantic proxy, the routing footprint is clearly sublinear and remains so at large routing budgets, with controls that tie the effect to structured angular signal. Second, the same fixed routing scaffold partially transfers to a small trainable language model without a learned gate matrix. At the same time, the trainable experiments show that the specific Hopf-derived assignment does not outperform a randomly permuted fixed routing map after end-to-end learning. The appropriate conclusion is therefore limited and direct: fixed geometric routing is viable as a scaffold, but the current evidence does not establish a geometry-specific advantage in trained language models.

References

Kawin Ethayarajh. How contextual are contextualized word representations? comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Conference on Empirical Methods in Natural Language Processing*, 2019.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23:1–39, 2022.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Chen, Gael Fried, Katherine Mahowald, et al. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021.

Mike Lewis, Luke Zettlemoyer, Edouard Grave, Armand Joulin, and Shruti Bhosale. BASE Layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning*, 2021.

Mitchell P. Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2):313–330, 1993.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016. Introduces WikiText-2 and WikiText-103 benchmarks.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. *arXiv preprint*, 2025.

A Detailed Experimental Setup

This appendix provides the supporting details needed to audit the main-text claims without widening them.

Table 2: Exact settings for the toy language-model experiments.

Hyperparameter	Value
Layers	2
Hidden dimension	128
Attention heads	4
Vocabulary size	5000
Sequence length	32
Experts	64
Expert dimension	128
Training steps	4000
Datasets	Penn Treebank, WikiText-2
Conditions	BASELINE, HOPF, PERMUTED

B Semantic Proxy Controls and Robustness

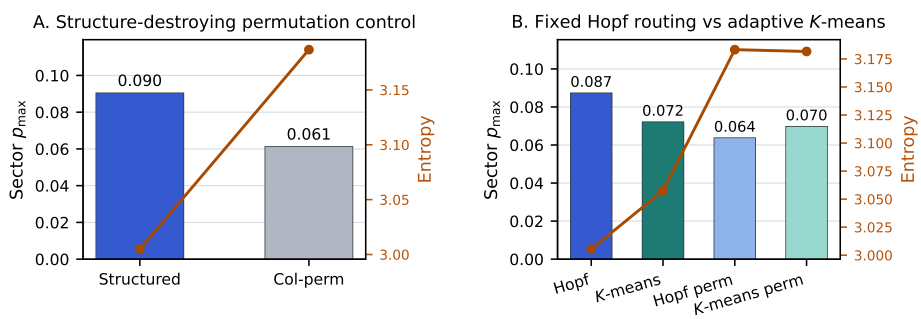

Figure 4: Proxy-side controls at appendix scale. The same two controls shown in the main text are repeated here for readability next to the detailed support tables.

Table 3: Structured-versus-permuted proxy control. The concentration effect weakens when the embedding coordinates are permuted before routing.

Proxy condition	Runs	Sector $p_{\max}$	Sector entropy	Test MSE
Structured	4	0.090	3.005	0.0040
Column-permuted	4	0.061	3.187	0.0040

Table 4: Hopf-versus-K-means control on the semantic proxy. Fixed Hopf routing is more concentrated than adaptive Euclidean K-means on the same structured embeddings.

Routing family	Runs	Sector $p_{\max}$	Sector entropy	Test MSE
Hopf	2	0.087	3.005	0.0040
K-means	2	0.072	3.058	0.0040
Hopf permuted	2	0.064	3.183	0.0040
K-means permuted	2	0.070	3.182	0.0040

B.1 Normalization robustness

Table 5: Scaling-law robustness across L1, L2, L3, and L4 normalization. The fitted Hopf exponent changes only minimally across these variants.

Normalization	Hopf exponent	Permuted exponent	Base exponent
L1	0.572	0.809	0.916
L2	0.572	0.814	0.916
L3	0.572	0.815	0.916
L4	0.572	0.816	0.916

C Language-Model Support Tables

Table 6: Compact summary of the main null result in the trainable language model.

Dataset	HOPF / BASE	HOPF - PERM	HOPF eff. paths	PERM eff. paths
PTB	1.081	+0.13	46.0	45.8
WT2	1.081	-0.03	40.9	42.3

Table 7: Cross-dataset summary for the trainable-model results.

Metric	PTB	WT2
HOPF / BASELINE PPL ratio	1.081×	1.081×
HOPF - PERMUTED PPL	+0.13	-0.03
HOPF effective-path ratio	1.39×	1.56×
BASELINE eff. paths	43.2	35.4
HOPF eff. paths	46.0	40.9
PERMUTED eff. paths	45.8	42.3

Table 8: Auxiliary-loss control supporting the choice of no-aux top-1 routing as the learned comparator. The load-balancing objective drives the learned router toward near-uniform usage.

Condition	Val. PPL	Eff. paths	Params
BASELINE	154.55	44.0	5.66M
LEARNED_SPARSE	154.78	62.8	5.66M
HOPF	165.59	47.7	5.64M
PERMUTED	165.25	47.1	5.64M

Table 9: Combined control and null-result summary used to interpret the trainable-model evidence.

Case	HOPF / BASE	HOPF - PERM	Aux eff. paths	Interpretation
PTB	1.081	+0.13	–	Hopf and permuted are effectively indistinguishable on PTB quality.
WT2	1.081	-0.03	–	The same null result replicates on WT2.
PTB_aux_control_screen	1.071	+0.34	62.8	Load-balancing aux loss drives routing toward near-uniform usage, so no-aux top-1 is the correct paper comparator.

Table 10: Detailed Penn Treebank language-model results.

Condition	Val. PPL	Eff. paths	$64/\text{eff}_b$	Params
BASELINE	152.26	43.19	1.48×	5.66M
HOPF	164.54	45.96	1.39×	5.64M
PERMUTED	164.41	45.81	1.40×	5.64M

Table 11: Detailed WikiText-2 language-model results.

Condition	Val. PPL	Std. PPL	Eff. paths	$64/\text{eff}_b$
BASELINE	105.85	1.57	35.37	1.81×
HOPF	114.45	1.14	40.94	1.56×
PERMUTED	114.48	0.87	42.28	1.51×